Layered Representations from a Single Image

Semantic Segmentation, Monocular Depth Ordering and Intrinsic Appearance Decomposition for Editable Scene Layers

Course Project Report

DEEP LEARNING FOR COMPUTER VISION

IITM BS Elective Course By

Dr Vineeth N Balasubramanian,

IIT Hyderabad

Source: IITM BS and NPTEL Course

Submitted by:

SUSHEEL KUMAR SHUKLA

22F1000835

Table of Contents

Abstract **2**
1. Introduction **3**
2. Problem Statement **3**
3. Objectives **4**
4. Relation to Course Concepts **4**
5. Proposed Methodology **4**
5.1 Overview of the Pipeline 4
5.2 Instance Segmentation 5
5.3 Background Extraction 5
5.4 Monocular Depth Estimation 5
5.5 Approximate Intrinsic Decomposition 5
5.6 Layer Construction and Recomposition 5
6. Implementation Details **7**
7. Experimental Evaluation **7**
7.1 Segmentation Quality 7
7.2 Depth Ordering 7
7.3 Reconstruction Fidelity 8
7.4 Editing Demonstrations 8
8. Limitations **8**
9. Future Work **8**
10. Conclusion **9**
References **9**

Abstract

A standard RGB photograph flattens a three-dimensional scene into a single grid of colour values, and by itself it says nothing about which pixels belong to which object, how far each object sits from the camera, or how much of a surface's appearance is due to its own colour versus the lighting falling on it. This project addresses that gap by converting a single input image into a stack of layered, editable representations. Each layer corresponds to one detected object (or the background), and carries its own transparency mask, an estimate of relative depth, and an approximate split of its appearance into albedo and shading.

The pipeline combines three pretrained components: a Mask R-CNN network for instance segmentation, a DPT/MiDaS monocular depth network for dense relative depth, and a lightweight Retinex-style filter that separates illumination from reflectance in the logarithmic intensity domain. The resulting RGBA layers can be reordered by depth, recomposited through standard alpha blending, edited individually (removed, brightened, or shifted), and combined into a simple depth-based parallax effect. The project does not train a new network from scratch; instead, its contribution lies in stitching together segmentation, depth and appearance estimation into one coherent, inspectable and editable scene representation, which is evaluated through reconstruction error and qualitative editing demonstrations.

1. Introduction

CNNs have made it possible to extract rich structure from a single photograph, objects can be localized and segmented, and depth can be estimated even without a stereo pair or a depth sensor. Individually, these capabilities are well studied and already covered across the Deep Learning for Computer Vision course: image representation and formation, convolutional feature extraction, transfer learning with pretrained backbones, and dense-prediction tasks such as segmentation.

This project builds exactly that combination. Starting from one RGB image, it produces a small set of RGBA (colour plus transparency) layers, one per detected object plus one for the background, each additionally tagged with a relative depth value and a rough albedo/shading split. Because every layer is self-contained, the set of layers can be manipulated the way a graphic designer would manipulate layers in an image-editing tool: hidden, reordered, brightened, or shifted, and then flattened back into a single image through standard alpha compositing. The project therefore turns a passive image into something closer to an editable scene description, using only pretrained models and simple image-processing steps, which keeps the implementation reproducible on a free Google Colab GPU instance.

2. Problem Statement

Given a single RGB image, the goal of this project is to produce an interpretable and re-composable layered representation of the scene depicted in it. The representation must separate the image into semantically meaningful regions — people, vehicles, animals, furniture, and a residual background — and must additionally order these regions by their approximate distance from the camera. Each region should also be assigned an approximate decomposition of its surface appearance into a reflectance (albedo) component and an illumination (shading) component. The final deliverable is a stack of RGBA layers that can be visualised independently, edited, reordered, or removed, and recombined to reconstruct a modified version of the original scene.

3. Objectives

- Build an end-to-end pipeline that converts one RGB image into a set of labelled, depth-ordered RGBA layers.
- Use a pretrained Mask R-CNN model to detect objects and produce per-instance masks and class labels.
- Use a pretrained monocular depth network (DPT/MiDaS) to obtain a dense relative depth map for the same image.
- Assign each object a single representative depth value (median depth over its mask) and use it to rank layers from near to far.
- Approximate an intrinsic image decomposition (albedo and shading) for every layer using a Retinex-inspired, log-domain Gaussian filtering approach.
- Recompose the original image from its layers via alpha compositing and measure the reconstruction error quantitatively (MAE, MSE).
- Demonstrate practical layer-level edits: object removal, brightness adjustment, and a depth-based parallax effect.

4. Relation to Course Concepts

Every stage of the pipeline maps onto material covered in the Deep Learning for Computer Vision course, which progresses from basic image representation through convolutional architectures to segmentation and, later, generative and editing applications. The table below summarises this mapping so that the connection between the project and the syllabus is explicit rather than incidental.

Project Component	Related Course Topic
RGB tensor representation of the input image	Image formation and representation
Convolutional feature extraction	CNN fundamentals
ResNet-50 backbone reused without retraining	Transfer learning with pretrained networks
Instance segmentation via Mask R-CNN	CNN-based segmentation (FCN / U-Net / Mask R-CNN)
Dense per-pixel depth prediction	Dense prediction tasks in deep vision
Layer construction and RGBA representation	Image representation and manipulation
Object removal, brightness edits, parallax	Image editing and generative applications

5. Proposed Methodology

5.1 Overview of the Pipeline

The system takes one RGB image as input and processes it through two parallel branches that are later merged. The first branch performs instance segmentation to identify discrete objects; the second branch estimates a dense depth map for the whole scene. The per-object depth is obtained by pooling the depth branch's output within each mask from the segmentation branch. Once every object (and the background) has an associated mask and a depth value, the objects are sorted by depth and separately passed through an appearance-decomposition step that splits their colours into an albedo layer and a shading layer. The final output is a stack of RGBA images, ordered from nearest to farthest, each carrying its class label, confidence score, mask, depth value, and albedo/shading estimate.

5.2 Instance Segmentation

A Mask R-CNN model with a ResNet-50 Feature Pyramid Network backbone, pretrained on a large object-detection dataset, is used without any fine-tuning. For each input image it returns a set of candidate instances, each with a bounding box, a class label, a confidence score, and a soft segmentation mask. Detections below a confidence threshold of 0.60 are discarded to avoid noisy, low-confidence layers. The remaining soft masks are binarised at a 0.5 probability cut-off to obtain clean, per-object alpha masks.

5.3 Background Extraction

Rather than relying on a separate 'stuff' segmentation network, the background is defined simply as the set of pixels that do not fall inside any accepted object mask. Taking the union of all object masks and inverting it gives a background mask directly, which is treated as an additional layer placed conceptually behind all detected objects.

5.4 Monocular Depth Estimation

A pretrained DPT/MiDaS depth network estimates a dense depth value for every pixel in the image. Because monocular depth networks generally produce depth only up to an unknown scale and shift, the raw output is min-max normalised to the range 0–1 rather than being interpreted as a metric distance. For every detected object (and for the background) the median normalised depth over the corresponding mask is computed, since the median is far less sensitive than the mean to noisy or mis-segmented pixels along object boundaries. Sorting all layers by this median depth value yields the near-to-far ordering used for compositing.

5.5 Approximate Intrinsic Decomposition

To add an appearance dimension to each layer, the project approximates the classical intrinsic-image model, in which an observed image is treated as the product of a reflectance (albedo) term and an illumination (shading) term. Working in the logarithmic domain converts this multiplicative relationship into an additive one, and a large-kernel Gaussian blur of the log-intensity image is used as a simple estimate of the low-frequency shading component; dividing the original image by this estimated shading then yields the albedo. This is a well-known simplification rather than a learned or physically exact decomposition, and the report is explicit that the resulting albedo and shading images should be read as approximate appearance factors, since a single image alone cannot fully disambiguate reflectance from illumination.

5.6 Layer Construction and Recomposition

Each accepted object, plus the background, is written out as a four-channel RGBA image: the colour channels are copied directly from the original photograph, and the alpha channel is set from the corresponding binary mask. A matching pair of albedo and shading images is stored alongside each RGBA layer for inspection. To verify that no information beyond what is expected has been lost, the full set of depth-ordered layers is recombined using standard 'over' alpha compositing, and the resulting reconstruction is compared pixel-by-pixel against the original input using mean absolute error and mean squared error.

6. Implementation Details

The pipeline is implemented in Python within a Google Colab notebook so that it can run on a free GPU instance without any local setup. PyTorch and torchvision supply the Mask R-CNN model and its pretrained weights, while the Hugging Face Transformers library supplies the DPT depth-estimation model and its image processor. NumPy, OpenCV, SciPy and Pillow handle the numerical image manipulation (mask thresholding, Gaussian filtering for the shading estimate, and RGBA compositing), and Matplotlib is used to visualise segmentation overlays, depth maps, individual layers and the final reconstructions. Pandas is used to keep a running metadata table of each layer's class, confidence, and depth rank, which is exported as a CSV file alongside a JSON metadata summary and the individual PNG layers, all packaged into a single downloadable ZIP archive at the end of the run.

The overall processing order in the notebook is:

(1) load and display the input image,
(2) run Mask R-CNN and filter detections by confidence,
(3) derive the background mask from the union of object masks,
(4) run the depth model and normalise its output,
(5) compute per-layer median depth and sort layers accordingly,
(6) estimate albedo and shading for the whole image and mask them per layer,
(7) write out RGBA layers and appearance images with accompanying metadata, and
(8) recompose, evaluate, and demonstrate edits (removal, brightness change, and depth-based parallax).

7. Experimental Evaluation

The pipeline is evaluated along three axes: the quality of the underlying segmentation, the plausibility of the recovered depth ordering, and the fidelity of the final image reconstruction. A small custom test set spanning indoor, outdoor and street scenes is recommended so that the pipeline is exercised on scenes with varying numbers of objects, occlusion levels, and lighting conditions.

7.1 Segmentation Quality

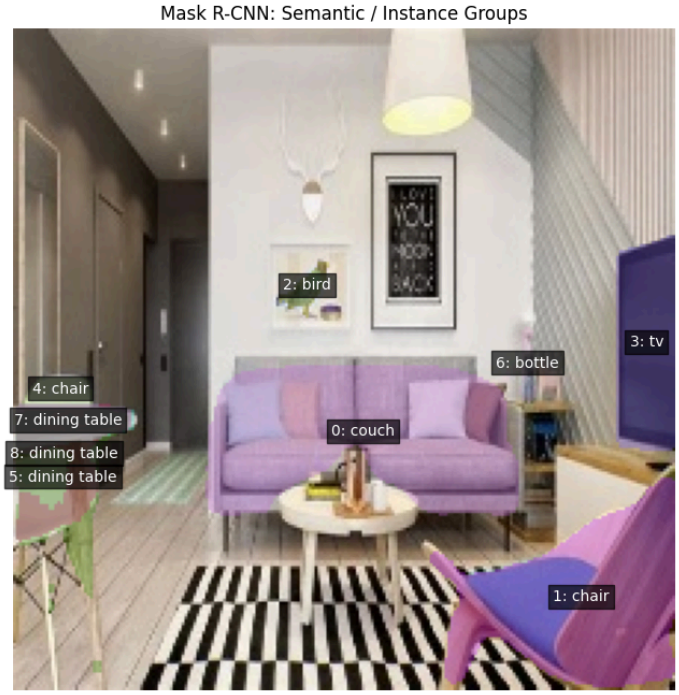
For each test image, the number of accepted detections and their mean confidence score are recorded.

Near → far ordering:

	depth_rank_near_to_far	original_index	semantic_class	confidence	median_depth	mean_depth	area_pixels	layer_type
0	1	1	chair	0.989893	0.982177	0.843104	3507	object
1	2	5	dining table	0.899893	0.426825	0.406383	1435	object
2	3	8	dining table	0.641894	0.422081	0.376983	946	object
3	4	3	tv	0.958587	0.383824	0.380891	1391	object
4	5	7	dining table	0.822803	0.327927	0.319770	513	object
5	6	4	chair	0.935642	0.265963	0.257806	211	object
6	7	0	couch	0.992629	0.153793	0.158664	4325	object
7	8	6	bottle	0.896763	0.124195	0.123018	104	object
8	9	-1	background	1.000000	0.117197	0.184345	39001	background
9	10	2	bird	0.975950	0.112743	0.112600	132	object

7.2 Depth Ordering

For a representative image, the layers are ranked from nearest to farthest using their median normalised depth.

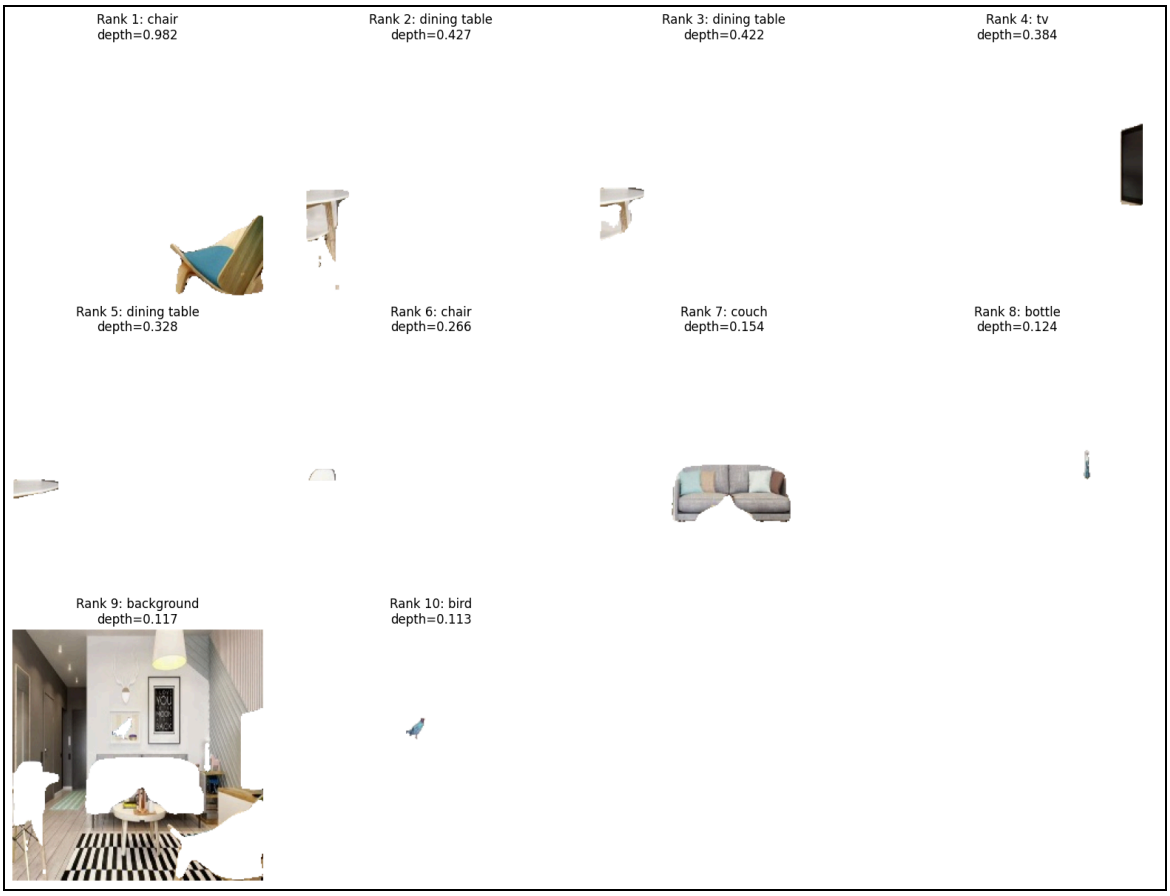


7.3 Reconstruction Fidelity

The layered representation is only useful if it can be flattened back into something close to the original photograph. Mean absolute error and mean squared error between the original image and the alpha-composited reconstruction quantify this. Because layers are constructed directly from the original pixels with hard binary masks, the reconstruction error is expected to be very low except at soft object boundaries where the segmentation mask was thresholded.

7.4 Editing Demonstrations

Beyond reconstruction, the project demonstrates that the layered representation is directly editable without touching the underlying neural networks: an object layer can be dropped from the composite entirely (object removal), its RGB values can be scaled to change its brightness, or it can be shifted horizontally in proportion to its depth to create a simple parallax effect where nearer layers move more than farther ones. These edits operate purely on the stored RGBA layers and require no re-inference, which is the main practical benefit of building a layered representation in the first place.



8. Limitations

- Occlusion: partially hidden objects can produce incomplete or merged masks from Mask R-CNN.
- Depth ambiguity: monocular depth output is relative, not metric, so absolute distances cannot be recovered.
- Intrinsic ambiguity: separating albedo from shading using a single image is an inherently under-constrained problem, so the decomposition is only approximate.

- Background quality: treating the background as 'everything not detected' is simple but not equivalent to a dedicated stuff-segmentation model.
- Thin structures: fine objects such as wires, branches, or fences are prone to segmentation errors.

9. Future Work

- Replace Mask R-CNN with a more modern panoptic model such as Mask2Former for combined 'things and stuff' segmentation.
- Swap the Retinex-style approximation for a learned intrinsic-decomposition network.
- Use a depth model with sharper object-boundary accuracy to reduce edge artefacts in the layer masks.
- Apply inpainting to fill in regions left empty after an object is removed, rather than exposing whatever lies behind it.
- Extend the parallax effect toward a small pseudo-3D camera-motion demonstration using depth-based reprojection.
- Explore a diffusion-based generative model for completing occluded or newly exposed regions.

10. Conclusion

This project shows that a single RGB photograph can be turned into a structured, editable scene representation using only pretrained deep-learning components and a small amount of classical image-processing. By combining instance segmentation, monocular depth estimation and an approximate intrinsic decomposition, each detected object is expressed as a self-contained RGBA layer carrying its identity, mask, relative depth, and rough appearance split. These layers can be reordered, edited, and recombined without any further neural network inference, and the resulting reconstruction closely matches the original image. The main contribution is not a new model but a practical integration of existing course concepts — CNN-based segmentation, dense depth prediction, and image editing — into one coherent and inspectable pipeline that is feasible to run and evaluate within a single Colab session.

References

[1] Deep Learning for Computer Vision from IITM BS course and NPTEL course materials — https://dl4cv-nptel.github.io/DL4CVBK/intro.html

[2] He, K., Gkioxari, G., Dollár, P., & Girshick, R. Mask R-CNN. ICCV 2017.

[3] Ranftl, R. et al. Vision Transformers for Dense Prediction (DPT). ICCV 2021.

[4] Land, E. H., & McCann, J. J. Lightness and Retinex Theory. Journal of the Optical Society of America, 1971.